

Claude Code Token Usage Assessment

Repository: DataLoaderPlatform | Date: 2026-08-24 | Audience: Engineering Team

STATIC INSTRUCTION CORPUS

~24.5k tokens

API + DESIGN DOCS CORPUS

~220.9k tokens

EXPECTED SAVINGS AFTER OPTIMIZATION

45% to 80%

Executive Summary

High token usage is structural, not accidental. The current workflow enforces multi-stage agent orchestration with repeated ingestion of large instruction and documentation context. This compounds cost per loader task, especially when review/test loops are triggered.

Measured Context Load

Source	Measured Size	Approx Tokens	Why It Matters
CLAUDE.md + 8 agent prompt files	97,986 bytes	~24,496	Heavy baseline loaded across many stages.
docs/apis/*.md	305,917 bytes	~76,479	Large field references repeatedly consulted.
docs/design/*.md	577,761 bytes	~144,440	Design docs are extensive and frequently re-read.

Token approximation used: bytes / 4 (practical planning heuristic).

Primary Drivers of Token Spend

Critical

Mandatory multi-stage orchestration

- Most loader tasks route through manager and full specialist chain.
- Each stage adds prompt + artifact + summary overhead.

Critical

Repeated ingestion of large static guidance

- Global conventions are rich and repeated across agent prompts.
- Instruction payload is paid repeatedly across stage loops.

High

Re-reading large API and design documents

- Documentation-first flow is valuable but expensive when over-applied.
- Same large docs are often re-consulted by multiple agents for one task.

High

Loop amplification (review/test/fix)

- Quality loops are correct, but broad context and long summaries increase cost per loop.
- Solution-wide test defaults further increase loop footprint.

Proposed Solutions and Expected Savings

Proposal	Process Change	Expected Savings
Fast lane vs full lane routing	Default to Coder -> Reviewer -> Tester for non-schema/non-API-shape changes; full chain only when needed.	Existing loader changes: 60% to 80%
Prompt compression to checklist style	Shorten CLAUDE + agent prompts; remove duplicated narrative while retaining hard quality gates.	All scenarios: 20% to 35% incremental
Scoped testing default	Run loader-scoped tests first; escalate to full-solution tests only for cross-cutting changes.	Review/test loops: 30% to 50%
Response-size discipline	Structured short outputs (files changed, decisions, risks, pass/fail); avoid large inline dumps.	Loop stages: 10% to 25%

Scenario impact overview

New loader build (estimated 45% to 65% savings)



Existing loader enhancement/bugfix (estimated 60% to 80% savings)



Review/test loops (estimated 30% to 50% savings)



Implementation Readiness

A detailed execution playbook is already prepared for post-loader implementation:

- [docs/superpowers/plans/2026-08-24-claude-token-usage-optimization-playbook.md](#)

This includes sequence, guardrails, validation checks, and rollback guidance.

Prepared by: GitHub Copilot (GPT-5.3-Codex) | Method: repository instruction audit + measured file-size token approximation